

Prompt 1b — Corrección de F1a antes de acumular serie

Urgente: aplicar ANTES de que el timer corra. El colector está desplegado y correcto en estructura, pero cuatro cosas harían que la serie que empiece a acumular esta madrugada nazca defectuosa. Dos de ellas no se pueden reparar después.

Repo: github.com/leanchoi/leanchoi, rama `claude/esquel-data-ecosystem-integration-qbxuv2`.
Hacé ``git pull --rebase`` primero: hay correcciones a `specs/config/rutas_muestreo.json`, `specs/sql/01_air_schema.sql` y `docs/01-motor-aereos.md`.

El despliegue de F1a está bien hecho. Esto NO es un rediseño: son cuatro correcciones acotadas. Aplicalas y volvé a dejar el timer activo.

=====

P0-1 – Persistir el payload crudo (NO SE PUEDE REPARAR DESPUÉS)

=====

`parse.py` parsea en memoria y nada escribe la respuesta original. Con un parser estrenado ayer –que ya nos mostró un fallo por Flybondi– eso significa que si aparece un segundo bug, los datos de esos días son IRRECUPERABLES.

- Escribir la respuesta cruda en bronce ANTES de parsear, comprimida, con el `query_id` como nombre y referencia en `raw_ref` de la bitácora.
- Retención 90 días (`global.retencion_crudo_dias` en la config). Es JSON, pesa poco; medí el tamaño real la primera semana y ajustá.
- Regla: si el parser falla, el crudo tiene que estar igual. Guardar primero, parsear después. Nunca al revés.

Toda la capa bronce existe para poder reprocesar sin volver a scrapear. Sin el crudo, no hay capa bronce: hay una capa plata con otro nombre.

=====

P0-2 – One-way por sentido, no round trip con retorno fijo

=====

Diagnóstico de tu propia verificación en vivo: las 3 consultas que dieron `sin_resultados` tienen MARTES en un tramo; las 2 que funcionaron, no. Esquel vuela diario salvo martes, así que no era un bug tuyo – era un error de la spec.

Por qué el round trip con offset fijo está mal, en orden creciente de gravedad:

1. Cobertura: falla si la ida es martes o si la vuelta cae martes (ida en sábado). ~29% de las fechas ancla perdidas, SIEMPRE los mismos días de semana. Y como los viernes zafan (`viernes+3 = lunes`), lo que se pierde son justo los puentes y los hitos: las fechas de mayor valor.
2. Curva de anticipación: un precio de round trip está atado a `DOS flight_date`

y no se puede desagregar. La serie de t queda contaminada con el precio de t+3. El indicador central del sistema deja de ser computable limpio.

3. TTCI: con retorno fijo a +3 solo existe TTCI para N=3 noches, cuando N lo elige el usuario en el navegador.

Cambios:

- La observación atómica es ONE-WAY. `bidireccional: true` en la config expande cada ruta en dos consultas (o->d y d->o). Ambos sentidos son serie propia, no un accesorio del TTCI.
- $TTCI(N) = F_{ida}(t) + F_{vuelta}(t+N)$ se compone en el navegador, para cualquier N.
- El round trip queda como CALIBRACIÓN: ~8 consultas semanales sobre BUE-EQS y BUE-BRC con 3/4/7 noches, evitando días sin servicio, para estimar el factor de descuento RT por aerolínea. Ver `calibracion\_roundtrip` en la config.

=====

P1-1 – Distinguir sin_servicio de sin_resultados

=====

Hoy todo cero cae en sin_resultados. Pero "el martes no vuela" y "la consulta no devolvió nada y no sé por qué" son cosas opuestas: la primera es un DATO, la segunda es un HUECO.

- Nuevo estado sin_servicio en la bitácora (ya está en 01_air_schema.sql).
- Se clasifica sin_servicio cuando la respuesta es válida, trae cero itinerarios y el calendario de servicio conocido para esa ruta y día lo explica. El resto queda en sin_resultados.
- sin_servicio NO descuenta cobertura. Con ~2 de 7 días sin vuelo, mezclarlos deja la cobertura clavada en ~71% y la marca "preliminar" encendida para siempre – el sistema se auto-desacredita con datos correctos.
- Efecto lateral valioso: la acumulación de sin_servicio RECONSTRUYE el calendario de servicio real por ruta, que es justamente uno de los datos que no publica bien ninguna fuente de terceros.

=====

P1-2 – No truncar el plan en silencio

=====

schedule.py termina con `return plan\_total[:tope\_dia]`. Las consultas que quedan afuera desaparecen sin dejar rastro, así que el denominador de cobertura pasa a ser "lo que intenté" en vez de "lo que planifiqué": el sistema reporta 100% de cobertura mientras descarta trabajo.

- Las consultas por encima del tope se REGISTRAN en la bitácora como omitido_por_presupuesto (el estado ya existe) y no se ejecutan.
- Hoy no truncás (165 < 250), pero al sumar tiers 3-4, rolling y checkpoints en Flb vas a truncar todos los días. Arreglalo ahora que es barato.

=====

P2 – Reconciliar el generador de fechas ancla

=====

generar_fechas_ancla() agrega TODOS los días de octubre, 1-15 de noviembre y 5-ene a 20-feb. La config declara `muestras: 6` para esos hitos: son 6 fechas muestreadas dentro del rango, no el rango completo. De ahí salen 165 consultas donde la spec estimaba ~72.

- Implementé `muestras`: elegí N fechas del rango de forma ESTABLE (hash de la fecha, no random) para que la misma fecha se muestree todos los días y la curva de anticipación no tenga agujeros.
- Recién después expandí a ambos sentidos. Con el generador actual, duplicar por sentido te deja en ~330/día y reventás el tope.
- Presupuesto objetivo con la corrección: ~143 consultas/día para tiers 1-2 en ambos sentidos.

=====

P2 – Verificar que las latencias no sean caché

=====

174-471 ms por consulta es rápido para una búsqueda real de Google Flights (la spec estimaba ~1,5 s). Puede ser reutilización de conexión, pero verificalo: corrí la misma consulta dos veces separadas por 10 minutos y confirmé que el payload NO es byte-idéntico. Si lo es, estás leyendo caché y las series van a estar rezagadas sin que se note.

=====

CRITERIOS DE ACEPTACIÓN

=====

1. El crudo se persiste ANTES de parsear; raw_ref apunta a un archivo que existe. Probalo forzando una excepción en el parser: el crudo debe quedar.
2. Reprocesar bronce -> parseo reproduce exactamente la salida de la corrida.
3. Las consultas one-way por sentido generan filas para BUE->EQS y EQS->BUE por separado, con flight_date propio.
4. Un martes en EQS produce sin_servicio, no sin_resultados, y NO descuenta cobertura.
5. Con el tope bajado a 10 a propósito, las consultas excedentes aparecen en la bitácora como omitido_por_presupuesto.
6. El generador con `muestras: 6` produce 6 fechas por hito, estables entre corridas consecutivas.
7. Presupuesto diario reportado y dentro de 250.
8. La misma consulta repetida a 10 minutos devuelve payloads distintos.

Después de esto, las TRES NOCHES del criterio de aceptación original de Fla empiezan a contar. Reporté cobertura, RSS y duración de cada una.

